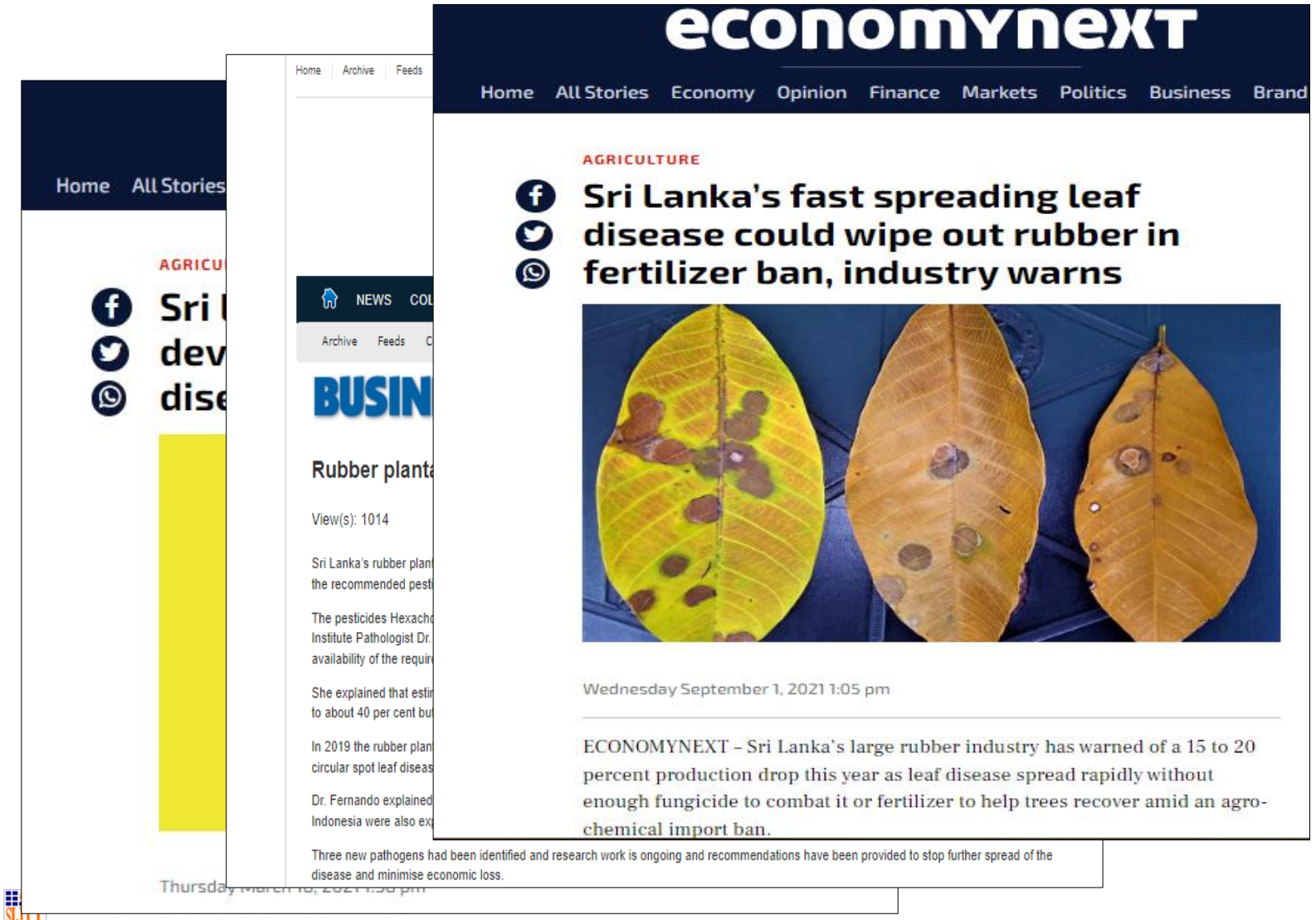
Identification of rubber leaf diseases

Imbulana Liyanage D.S.I.
IT19134772
SE Specialization

Research Problem



Pestalotiopsis
Collectrotrichum
caused
10% - 20%
Crop loss

50 Million LKR
Allocated for
research

Diseases had spread
to approximately
40, 000 Hectares
by 2019

Research problem

- Damage due to diseases
- Lack of expert knowledge
- Experiments are time consuming
- Unavailability of laboratory facilities
- Production cost is high

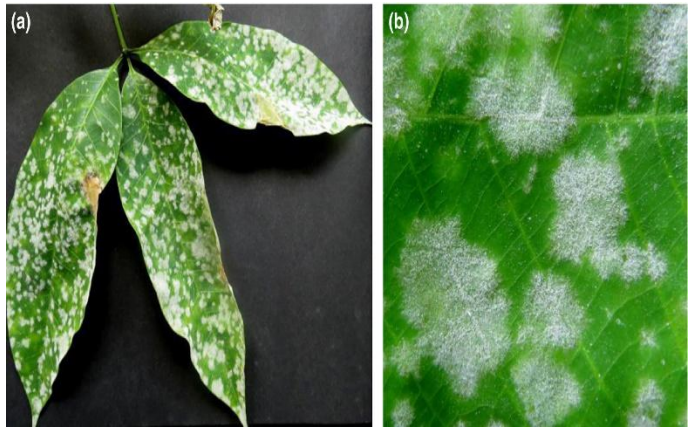
Background

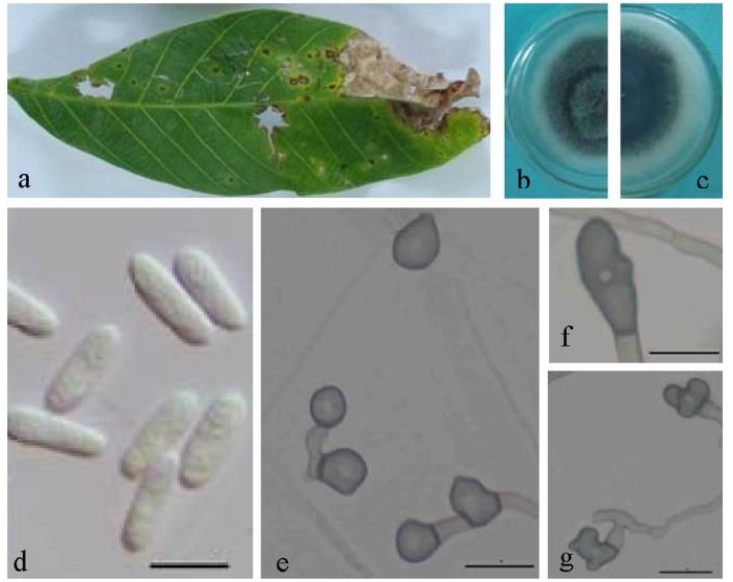
What are Leaf diseases?

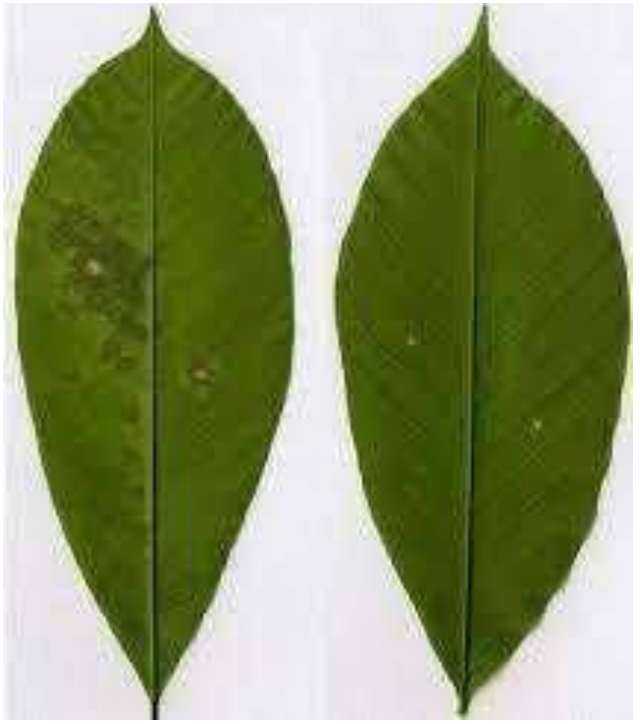
- Birdseye Leaf Disease
- Collectotrichum Leaf Disease
- Corynespora Leaf Fall Disease
- “New Leaf” Disease
- Oidium Leaf Disease
- Pestalotiopsis Leaf Disease

Symptoms

Treatments and Recommendations







Specific and Sub objectives

Specific Objective

- Develop an application that provides expert solutions for Identification of rubber leaf diseases

Sub Objectives

- Give a quick way to identify diseases though a photo of leaf
- Provide treatments and recommendations for the relevant disease
- Can control the diseases in the initial state
- Provide access the application in offline as well

Field Visits

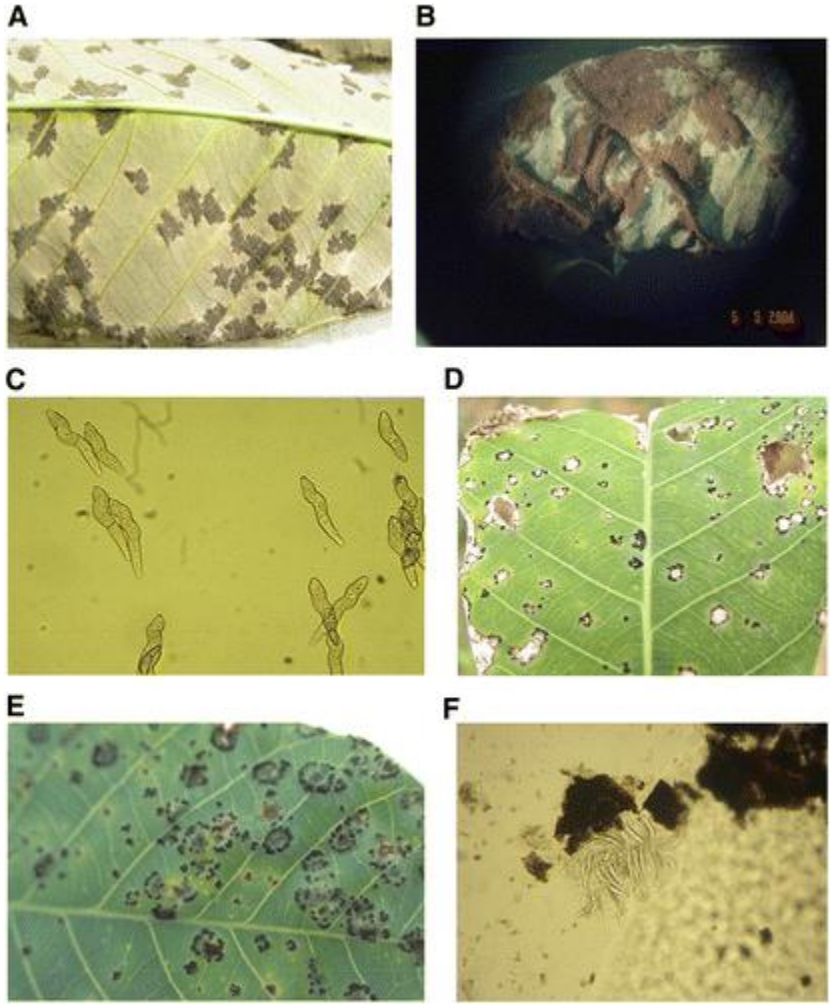
- Agalawatta Rubber Research Institute
- 2nd March 2022
- Padukka Rubber State
- 6th March 2022

Dataset

- Collected from rubber leaf samples by taking photos

500 images of ,

- Birdseye Leaf Disease
- Collectotrichum Leaf Disease
- Corynespora Leaf Fall Disease
- New Leaf Disease Leaf Disease
- Oidium Leaf Disease
- Pestalotiopsis Leaf Disease



Training Leaf Disease Recognition Model

- Trained models

Name	Accuracy
Sequential Model – 2 (CNN)	96.88%
ResNet	26%
VGG16	36%
VGG16 with Transfer Learning	25.94%
Sequential Model – 1	59%

Dataset

- Get reshape images to 256 x 256

```
BATCH_SIZE = 32
IMAGE_SIZE = 256
CHANNELS=3
```

```
dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "RubberLeafDiseases",
    seed=123,
    shuffle=True,
    image_size=(IMAGE_SIZE,IMAGE_SIZE),
    batch_size=BATCH_SIZE
)
```

```
Found 603 files belonging to 7 classes.
```

```
class_names = dataset.class_names
class_names
```

```
['Birdseye_leaf_disease',
 'Colletrotrichum',
 'Corynespora_leaf_fall_disease',
 'New_leaf_disease',
 'Oidium',
 'Pestalotiopsis',
 'random']
```

Function to Split Dataset

Dataset should be bifurcated into 3 subsets, namely:

1. Training: Dataset to be used while training
2. Validation: Dataset to be tested against while training
3. Test: Dataset to be tested against after we trained a model

```
def get_dataset_partitions_tf(ds, train_split=0.8, val_split=0.1, test_split=0.1, shuffle=True, shuffle_size=10000):
    assert (train_split + test_split + val_split) == 1

    ds_size = len(ds)

    if shuffle:
        ds = ds.shuffle(shuffle_size, seed=12)

    train_size = int(train_split * ds_size)
    val_size = int(val_split * ds_size)

    train_ds = ds.take(train_size)
    val_ds = ds.skip(train_size).take(val_size)
    test_ds = ds.skip(train_size).skip(val_size)

    return train_ds, val_ds, test_ds
```

```
train_ds, val_ds, test_ds = get_dataset_partitions_tf(dataset)
```

Cache, Shuffle, and Prefetch the Dataset

Shuffle function that it replaces to handle datasets that are too large to fit in memory.

Cache transformation can cache a dataset, either in memory or on local storage. This will save some operations (like file opening and data reading) from being executed during each epoch.

Prefect can be used to decouple the time when data is produced from the time when data is consumed.

```
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
val_ds = val_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
test_ds = test_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
```

Build the layer

- To improve model performance, it should normalize the image pixel value (keeping them in range 0 and 1 by dividing by 256). This should happen while training as well as inference. Hence we can add that as a layer in our Sequential Model.

```
resize_and_rescale = tf.keras.Sequential([
  layers.experimental.preprocessing.Resizing(IMAGE_SIZE, IMAGE_SIZE),
  layers.experimental.preprocessing.Rescaling(1./255),
])
```

Data Augmentation

- Data Augmentation is needed when it have less data, this boosts the accuracy of the model by augmenting the data.

```
data_augmentation = tf.keras.Sequential([
  layers.experimental.preprocessing.RandomFlip("horizontal_and_vertical"),
  layers.experimental.preprocessing.RandomRotation(0.2),
])
```

- Applying Data Augmentation to Train Dataset

```
train_ds = train_ds.map(
    lambda x, y: (data_augmentation(x, training=True), y)
).prefetch(buffer_size=tf.data.AUTOTUNE)
```

Model Architecture

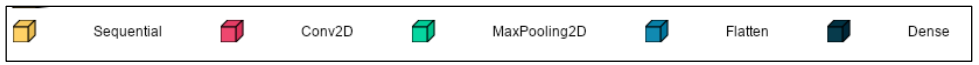
```
input_shape = (BATCH_SIZE, IMAGE_SIZE, IMAGE_SIZE, CHANNELS)
n_classes = 7

model = models.Sequential([
    resize_and_rescale,
    layers.Conv2D(32, kernel_size = (3,3), activation='relu', input_shape=input_shape),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64,  kernel_size = (3,3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64,  kernel_size = (3,3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(n_classes, activation='softmax'),
])

model.build(input_shape=input_shape)
```

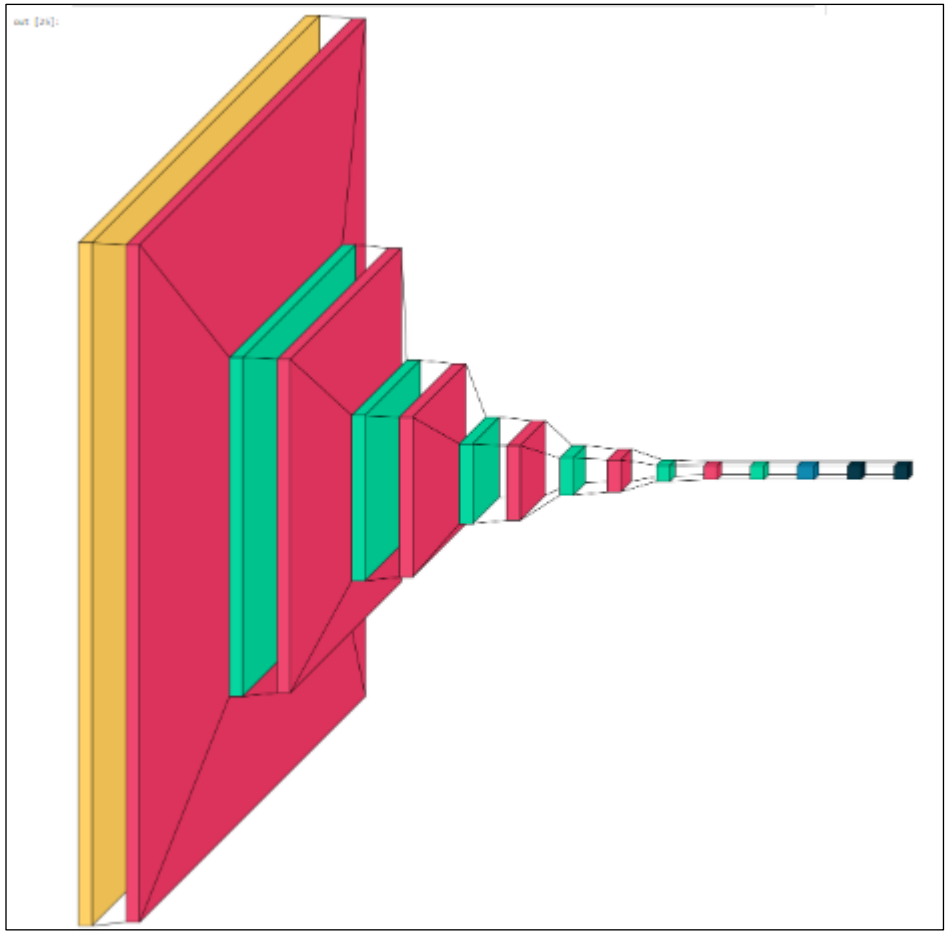
Model Summary

```
Model: "sequential_2"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 sequential (Sequential)     (32, 256, 256, 3)         0

 conv2d (Conv2D)             (32, 254, 254, 32)        896

 max_pooling2d (MaxPooling2D  (32, 127, 127, 32)       0
 )

 conv2d_1 (Conv2D)           (32, 125, 125, 64)        18496

 max_pooling2d_1 (MaxPooling  (32, 62, 62, 64)         0
 2D)

 conv2d_2 (Conv2D)           (32, 60, 60, 64)          36928

 max_pooling2d_2 (MaxPooling  (32, 30, 30, 64)         0
 2D)

 conv2d_3 (Conv2D)           (32, 28, 28, 64)          36928

 max_pooling2d_3 (MaxPooling  (32, 14, 14, 64)         0
 2D)

 max_pooling2d_3 (MaxPooling  (32, 14, 14, 64)         0
 2D)

 conv2d_4 (Conv2D)           (32, 12, 12, 64)          36928

 max_pooling2d_4 (MaxPooling  (32, 6, 6, 64)           0
 2D)

 conv2d_5 (Conv2D)           (32, 4, 4, 64)            36928

 max_pooling2d_5 (MaxPooling  (32, 2, 2, 64)           0
 2D)

 flatten (Flatten)           (32, 256)                 0

 dense (Dense)               (32, 64)                  16448

 dense_1 (Dense)             (32, 7)                   455

=================================================================
Total params: 184,007
Trainable params: 184,007
Non-trainable params: 0
```

Model Summary Visualization

```
import visualkeras
from PIL import ImageFont
font = ImageFont.truetype("arial.ttf", 12, encoding="unic")
visualkeras.layered_view(model, legend=True, spacing=50,font=font)
```





Compiling the Model

We use **Adam Optimizer,** SparseCategoricalCrossentropy for losses, accuracy as a metric

Adam optimizer involves a combination of two gradient descent methodologies.

```
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy']
)
```

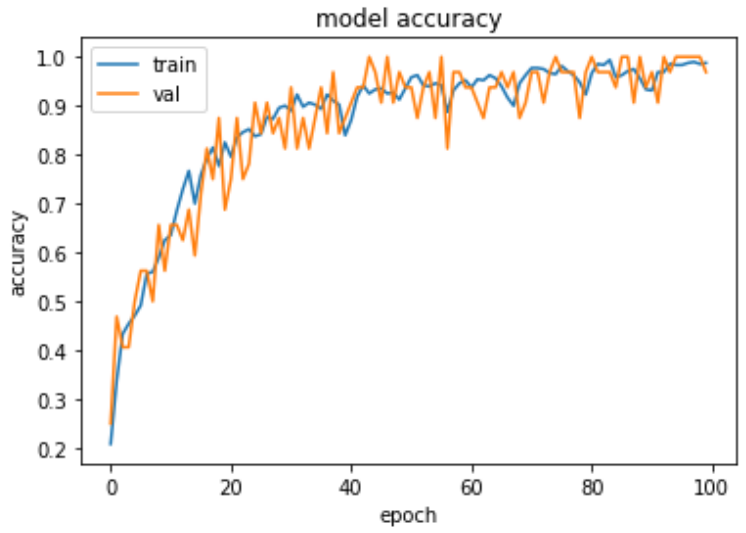
```
history = model.fit(
    train_ds,
    batch_size=BATCH_SIZE,
    validation_data=val_ds,
    verbose=1,
    epochs=100,
)
```

```
96/100
[==============================] - 36s 2s/step - loss: 0.0396 - accuracy: 0.9833 - val_loss: 0.0414 - val_accuracy: 1.0000
97/100
[==============================] - 37s 2s/step - loss: 0.0462 - accuracy: 0.9875 - val_loss: 0.0375 - val_accuracy: 1.0000
98/100
[==============================] - 36s 2s/step - loss: 0.0347 - accuracy: 0.9896 - val_loss: 0.0215 - val_accuracy: 1.0000
99/100
[==============================] - 36s 2s/step - loss: 0.0323 - accuracy: 0.9854 - val_loss: 0.0180 - val_accuracy: 1.0000
100/100
[==============================] - 37s 2s/step - loss: 0.0498 - accuracy: 0.9875 - val_loss: 0.0734 - val_accuracy: 0.9688
```

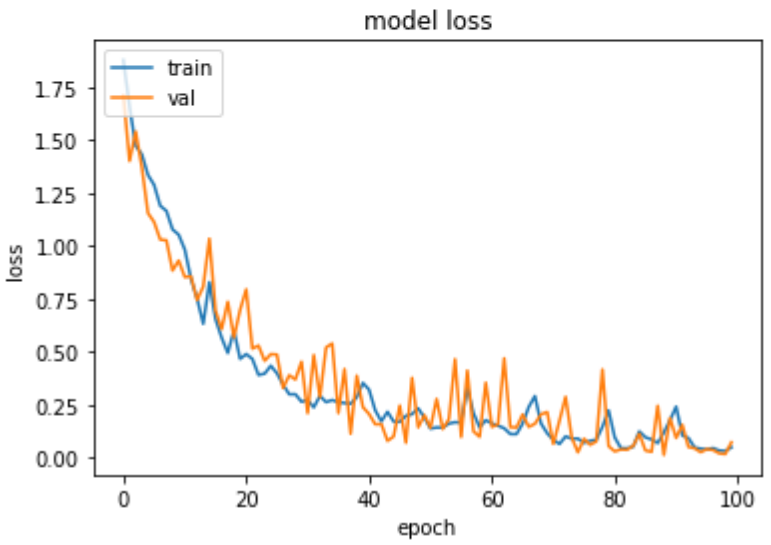
Model Training

Plot training accuracy and loss curves

```
#training accuracy
import matplotlib.pyplot as plt
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```




```
#loss curve
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```



Result Generation

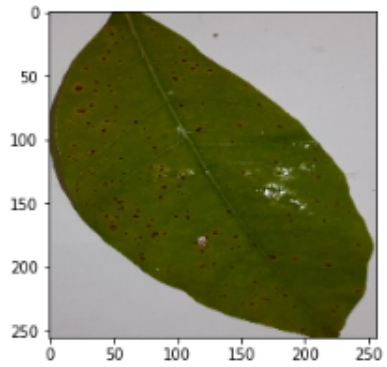
```python
def predict(model, img):
    img_array = tf.keras.preprocessing.image.img_to_array(images[i].numpy())
    img_array = tf.expand_dims(img_array, 0)

    predictions = model.predict(img_array)

    predicted_class = class_names[np.argmax(predictions[0])]
    confidence = round(100 * (np.max(predictions[0])), 2)
    return predicted_class, confidence
```

```python
plt.figure(figsize=(15, 15))
for images, labels in test_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))

        predicted_class, confidence = predict(model, images[i].numpy())
        actual_class = class_names[labels[i]]

        plt.title(f"Actual: {actual_class},\n Predicted: {predicted_class}.\n Confidence: {confidence}%")

        plt.axis("off")
```

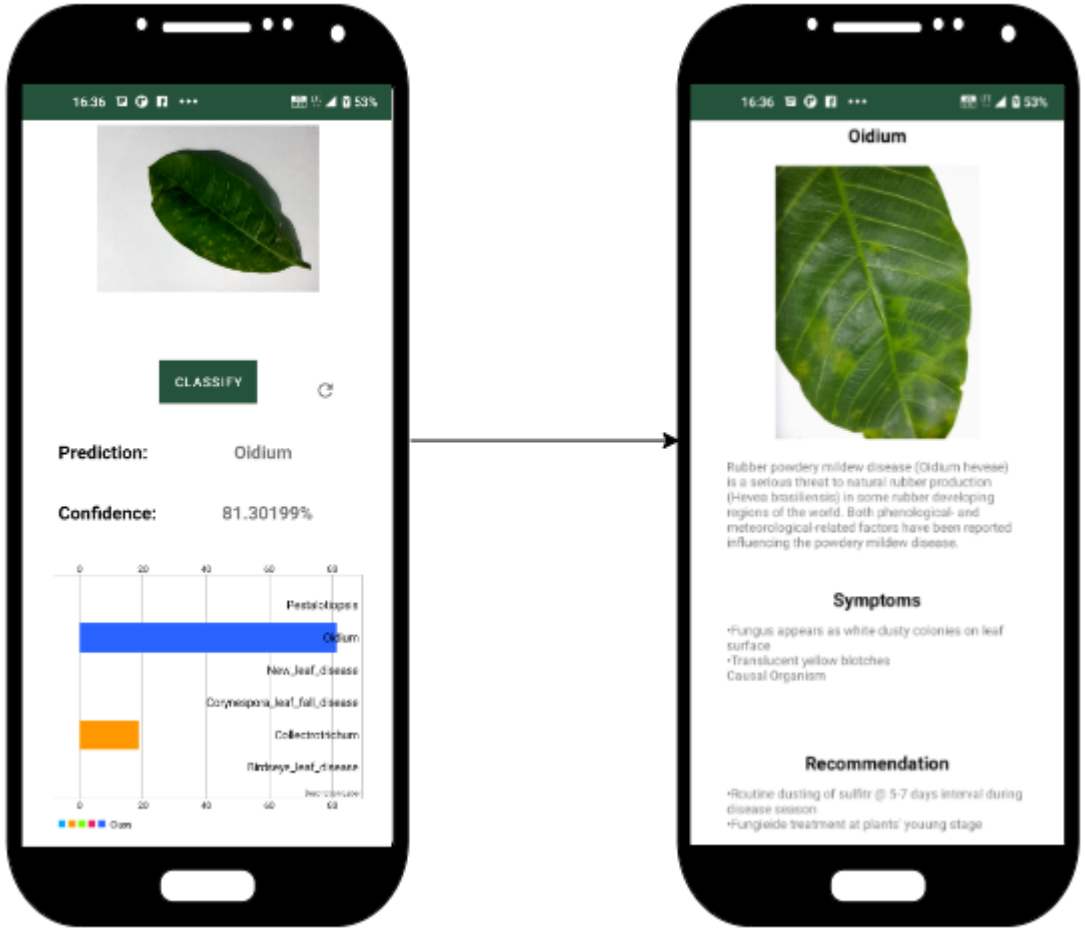

Actual: New_leaf_disease,
Predicted: New_leaf_disease.
Confidence: 99.99%
Actual: Birdseye_leaf_disease,
Predicted: Birdseye_leaf_disease.
Confidence: 99.91%
Actual: Colletotrichum,
Predicted: Colletotrichum.
Confidence: 100.0%
Actual: New_leaf_disease,
Predicted: New_leaf_disease.
Confidence: 99.89%
Actual: New_leaf_disease,
Predicted: New_leaf_disease.
Confidence: 99.96%
Actual: Birdseye_leaf_disease,
Predicted: Birdseye_leaf_disease.
Confidence: 98.71%

```
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
img = image.load_img("F:/SLIIT/4th Year/IUP/Practicals/RubberLeafDiseases/Birdseye_leaf_disease/20220302_221318.jpg",target_size=
img = np.asarray(img)
plt.imshow(img)
img = np.expand_dims(img, axis=0)
from tensorflow.keras.models import load_model
saved_model = load_model("RubberLeafDiseasesModel.h5")
output = saved_model.predict(img)

# print(output)
if output[0][0] > output[0][1]:
    print("Birdseye_leaf_disease")
else:
    print("Colletrotrichum")
```

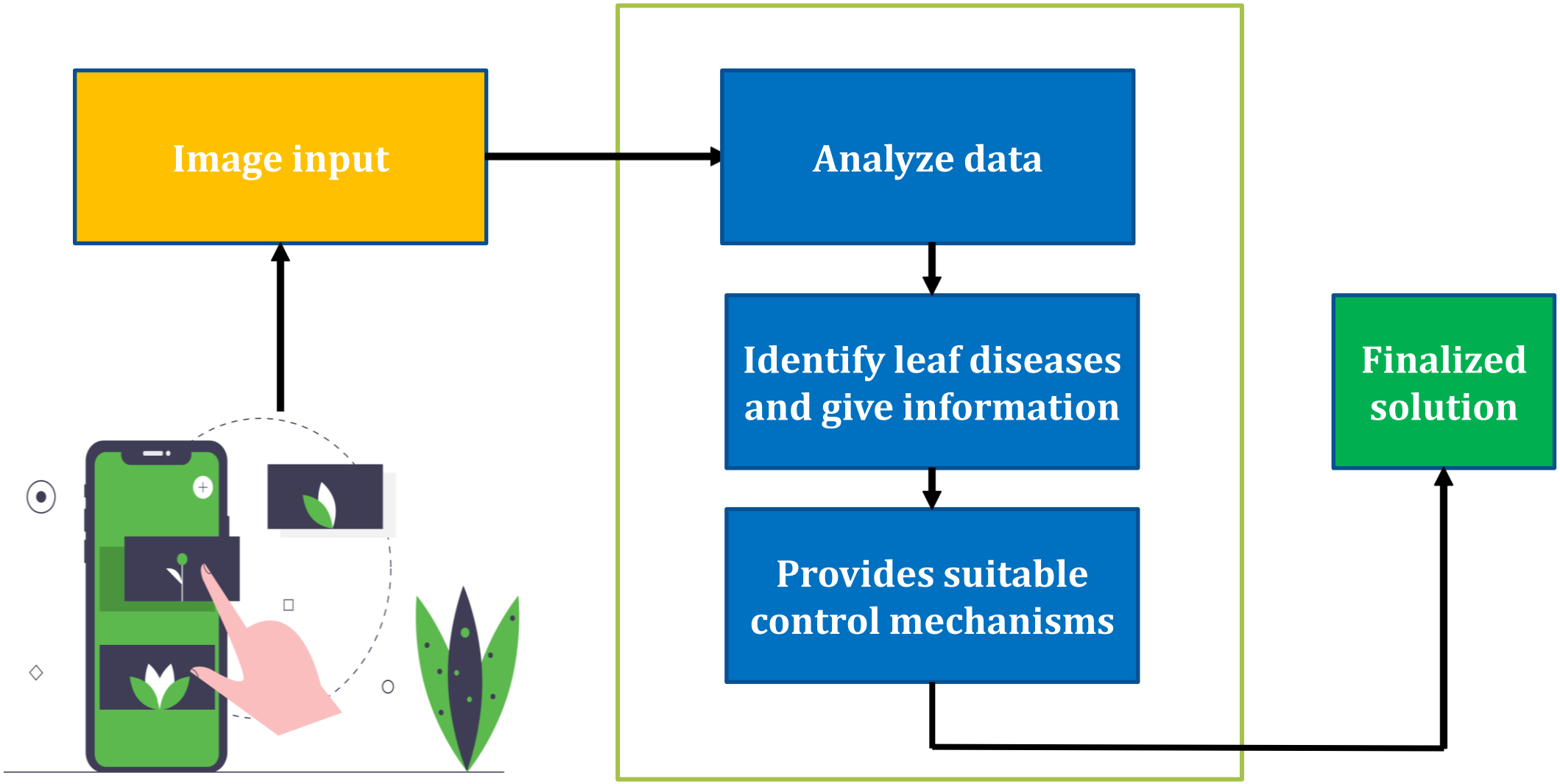
```
Birdseye_leaf_disease
```



Mobile application Development



System Diagram



References

[1] T. H. P. S. Fernando, “Economically important diseases of rubber plant,” in *Handbook of Rubber Vol. 1: Agronomy*, V.H.L. Rodrigo and P. Seneviratne, Eds., Rubber Research Institute of Sri Lanka, Dartonfield, Agalawatta,12200, Sri Lanka, 2021, pp. 118-153

[2] *Common Diseases of Rubber and Their Management,* Philippine Rubber Research Institute, Sanito, Ipil, Zamboanga Sibugay, Philippines, 2019

[3] T. H. P. S. Fernando, “Important diseases of Hevea brasiliensis uncommon or absent in Sri Lanka,” in *Handbook of Rubber Vol. 1: Agronomy*, V.H.L. Rodrigo and P. Seneviratne, Eds., Rubber Research Institute of Sri Lanka, Dartonfield, Agalawatta,12200, Sri Lanka, 2021, pp. 169-173

[4]“ Leaf Doctor: A New Portable Application for Quantifying Plant Disease Severity,” Sarah J. Pethybridge , School of Integrative Plant Science, Section of Plant Pathology and Plant-Microbe Biology, Cornell University, Geneva, NY 14456Scot C. Nelson , College of Tropical Agriculture and Human Resources, Department of Plant and Environmental Protection Sciences, University of Hawaii at Manoa, Honolulu, HI 96822